

Polimorfismo

Polimorfismo

Il concetto di polimorfismo in C++, con funzioni virtuali e override.

Cos'è il polimorfismo?

Immagina di avere una fattoria con cani, gatti e mucche. Vuoi far "parlare" tutti gli animali, uno per uno. Senza il polimorfismo, dovresti scrivere un ciclo separato per ogni tipo di animale.

Con il **polimorfismo**, puoi mettere tutti gli animali in una stessa lista e far sì che ognuno risponda al comando "emetti suono" a modo suo — automaticamente, senza controllare di che tipo è.

Polimorfismo significa "molte forme": lo stesso comando, comportamenti diversi a seconda dell'oggetto.

Il problema senza polimorfismo

```
class Cane {
public:
    void suona() { cout << "Bau!" << endl; }
};

class Gatto {
public:
    void suona() { cout << "Miao!" << endl; }
};

// Senza polimorfismo, serve una funzione per ogni tipo
void faiSuonare(Cane& c) { c.suona(); }
void faiSuonare(Gatto& g) { g.suona(); }
// E se aggiungi Mucca? Devi aggiungere un'altra funzione...
```

La parola chiave **virtual**

Aggiungendo **virtual** davanti a un metodo nella classe madre, dici al C++ di usare il comportamento della classe figlia, non della madre:

```
class Animale {
public:
    string nome;

    virtual void emettiSuono() { // virtual = usa il metodo della figlia
        cout << nome << " emette un suono." << endl;
    }
};

class Cane : public Animale {
public:
    void emettiSuono() override { // override = sovrascrive il metodo
base
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() override {
        cout << nome << " dice: Miao!" << endl;
    }
};
```

La parola **override** non è obbligatoria, ma è una buona abitudine: il compilatore ti avvisa se stai cercando di sovrascrivere un metodo che non esiste nella classe madre.

Il polimorfismo in azione

La "magia" si vede quando usi un **puntatore alla classe madre** per gestire oggetti di classi figlie diverse:

```
Animale* a1 = new Cane();    // puntatore ad Animale, ma è un Cane
Animale* a2 = new Gatto();   // puntatore ad Animale, ma è un Gatto

a1->nome = "Fido";
a2->nome = "Luna";

a1->emettiSuono();    // Fido dice: Bau!    (usa il metodo di Cane!)
a2->emettiSuono();    // Luna dice: Miao!    (usa il metodo di Gatto!)

delete a1;
delete a2;
```

Senza `virtual`, entrambi avrebbero chiamato il metodo di `Animale` (suono generico). Con `virtual`, il C++ sa di dover usare il metodo del tipo reale dell'oggetto.

Gestire una lista mista di oggetti

Questo è il punto di forza del polimorfismo: puoi mettere tipi diversi in una stessa lista e trattarli tutti allo stesso modo:

```

#include <iostream>
#include <vector>
using namespace std;

class Animale {
public:
    string nome;
    Animale(string n) : nome(n) {}
    virtual void emettiSuono() = 0;    // obbliga le classi figlie a
    implementarlo
    virtual ~Animale() {}             // distruttore virtuale (importante!)
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Bau!" << endl; }
};

class Gatto : public Animale {
public:
    Gatto(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Miao!" << endl; }
};

class Mucca : public Animale {
public:
    Mucca(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Mu!" << endl; }
};

int main() {
    vector fattoria;
    fattoria.push_back(new Cane("Fido"));
    fattoria.push_back(new Gatto("Luna"));
    fattoria.push_back(new Mucca("Bessie"));
    fattoria.push_back(new Cane("Rex"));

    // Un solo ciclo gestisce tutti i tipi di animali!
    for (Animale* a : fattoria) {
        a->emettiSuono();
    }

    // Libera la memoria
    for (Animale* a : fattoria) {
        delete a;
    }
}

```

```
    return 0;  
}
```

Output:

```
Fido: Bau!  
Luna: Miao!  
Bessie: Mu!  
Rex: Bau!
```

Classi astratte: il modello obbligatorio

Quando scrivi `= 0` dopo un metodo virtuale, quel metodo diventa **puro virtuale**: non ha corpo nella classe madre e ogni classe figlia è obbligata a implementarlo.

Una classe con almeno un metodo puro virtuale è **astratta**: non puoi creare oggetti direttamente da essa.

```

class Forma {
public:
    // Ogni forma DEVE implementare questi due metodi
    virtual double area() const = 0;
    virtual double perimetro() const = 0;

    // Questo metodo usa i precedenti (funziona grazie al polimorfismo)
    void stampa() const {
        cout << "Area: " << area() << ", Perimetro: " << perimetro() <<
endl;
    }
};

class Cerchio : public Forma {
private:
    double raggio;
public:
    Cerchio(double r) : raggio(r) {}
    double area() const override { return 3.14159 * raggio * raggio; }
    double perimetro() const override { return 2 * 3.14159 * raggio; }
};

class Rettangolo : public Forma {
private:
    double base, altezza;
public:
    Rettangolo(double b, double a) : base(b), altezza(a) {}
    double area() const override { return base * altezza; }
    double perimetro() const override { return 2 * (base + altezza); }
};

int main() {
    // Forma f; // ERRORE: Forma è astratta, non puoi creare oggetti diretti

    Cerchio c(5.0);
    Rettangolo r(4.0, 6.0);

    c.stampa(); // Area: 78.5397, Perimetro: 31.4159
    r.stampa(); // Area: 24, Perimetro: 20

    return 0;
}

```

Il distruttore virtuale

Quando usi il polimorfismo con puntatori alla classe madre, dichiara sempre il distruttore della classe madre come `virtual`. Altrimenti, quando elimini un oggetto con `delete`, viene chiamato solo il distruttore della madre — non quello della figlia:

```
class Base {
public:
    virtual ~Base() {           // distruttore virtuale
        cout << "Base distrutta" << endl;
    }
};

class Derivata : public Base {
public:
    ~Derivata() override {
        cout << "Derivata distrutta" << endl;
    }
};

Base* ptr = new Derivata();
delete ptr;    // con virtual: chiama Derivata prima, poi Base
               // senza virtual: chiama solo Base (bug!)
```